

TSwap Protocol Audit Report

Version 1.0

AYUBA WAPNEN WUBAL

October 22, 2026

Protocol Audit Report

AYUBA WAPNEN WUBAL

October 22, 2026

Prepared by: AYUBA WAPNEN WUBAL

Lead Auditors:

- AYUBA WAPNEN WUBAL

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` makes users overpay about 10x on exact-output swaps

    * [H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` lets users pay far more input tokens than expected
    * [H-3] `TSwapPool::sellPoolTokens` calls `swapExactOutput` with a pool-token amount as the WETH output, so users trade the wrong amount
    * [H-4] In `TSwapPool::_swap` the extra tokens paid out every 10 swaps break the `x * y = k` invariant and drain the pool
- Medium
    * [M-1] `TSwapPool::deposit` never applies its deadline, so deposits can execute after the deadline the user set
- Low
    * [L-1] `TSwapPool::LiquidityAdded` event emits its amounts in the wrong order
    * [L-2] `TSwapPool::swapExactInput` never assigns its named return value, so it always returns 0
    * [L-3] `TSwapPool::getPriceOfOnePoolTokenInWeth` hardcodes `1e18` as one pool token, giving meaningless prices for tokens that do not use 18 decimals
- Informationals
    * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
    * [I-2] `PoolFactory::constructor` lacks a zero-address check on `wethToken`
    * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`
    * [I-4] `TSwapPool::constructor` lacks zero-address checks on `poolToken` and `wethToken`
    * [I-5] Events are missing `indexed` fields
    * [I-6] Magic numbers should be replaced with named constants
    * [I-7] Dead and redundant code in `TSwapPool::deposit`
    * [I-8] NATSPEC is missing or incomplete on the swap functions
    * [I-9] `TSwapPool::swapExactInput` could be marked `external`
    * [I-10] `PoolFactory` imports a testing interface and uses an invalid SPDX identifier
    * [I-11] Missing validation in `getInputAmountBasedOnOutput` and the swap functions
    * [I-12] Price getters return fee-adjusted output rather than a spot price
    * [I-13] Fee-on-transfer and rebasing tokens are not supported

Protocol Summary

TSwap is a decentralized exchange built on the constant-product automated market maker model (`x * y = k`), similar to Uniswap V1. A `PoolFactory` contract creates one `TSwapPool` for each ERC20 token, paired with WETH. Liquidity providers deposit WETH and the pool token in the current pool ratio and receive liquidity (LP) tokens in return. Users swap between the two tokens, paying a 0.3% fee that accrues to liquidity providers. Liquidity providers burn LP tokens to withdraw their share of the reserves.

Disclaimer

The AYUBA WAPNEN WUBAL team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
./src/
|-- PoolFactory.sol
|-- TSwapPool.sol
```

- Solc version: 0.8.20
- Chain: any EVM chain
- Tokens: standard ERC20 tokens, paired with WETH
- Commit hash: e643a8d4c2c802490976b538dd009b351b1c8dda

Roles

- Liquidity Providers: deposit WETH and the pool token to earn fees, and withdraw by burning LP tokens.
- Users: swap between the pool token and WETH.

Executive Summary

The review combined manual code review, static analysis (Slither and Aderyn), compiler warnings, and a stateful fuzzing suite built with Foundry that checks the constant-product invariant. Each High, Medium, and Low finding below includes a proof of code that can be run with `forge test`.

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	3
Info	13
Total	21

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` makes users overpay about 10x on exact-output swaps

Description: `getInputAmountBasedOnOutput` returns `((inputReserves * outputAmount )* 10000)/ ((outputReserves - outputAmount)* 997)`. For a 0.3% fee the multiplier should be `1000`, which is what `getOutputAmountBasedOnInput` uses. With `10000` the required input is about ten times too high, so `swapExactInput` and `swapExactOutput` price the same trade very differently.

Impact: Users of `swapExactOutput`, and of `sellPoolTokens`, which calls it, pay roughly ten times the correct input amount. About 90% of what they pay is excess that stays in the pool. Because `swapExactOutput` has no maximum-input check (see [H-2]), users cannot detect or cap the overpayment.

Proof of Concept: In a pool with 100e18 pool tokens and 100e18 WETH, buying 1 WETH should cost about 1.013e18 pool tokens. With the bug it costs about 10.13e18.

Proof Of Code

Place the following into `TSwapPool.t.sol`.

```
function testFlawedSwapExactOutput() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 expectedInput = (100e18 * 1e18 * 1000) / ((100e18 - 1e18) *
        997);

    vm.startPrank(user);
    poolToken.mint(user, 100e18);
    poolToken.approve(address(pool), type(uint256).max);
    uint256 balanceBefore = poolToken.balanceOf(user);
    pool.swapExactOutput(poolToken, weth, 1e18, uint64(block.timestamp)
       );
    vm.stopPrank();

    uint256 spent = balanceBefore - poolToken.balanceOf(user);
```

```
18      assertGt(spent, expectedInput * 9); // user paid about 10x the
            correct amount
19  }
```

Recommended Mitigation: Use `1000` and define the fee values as named constants so the two pricing functions cannot drift apart.

```
1  - return ((inputReserves * outputAmount) * 10000) / ((outputReserves -
       outputAmount) * 997);
2  + return ((inputReserves * outputAmount) * 1000) / ((outputReserves -
       outputAmount) * 997);
```

[H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` lets users pay far more input tokens than expected

Description: `swapExactOutput` computes the required input from the pool's current reserves and pulls that amount from the caller, with no upper bound. `swapExactInput` takes a `minOutputAmount`, but `swapExactOutput` has no equivalent `maxInputAmount`. Anything that moves the reserves between submission and execution changes what the user pays.

Impact: A user who requests an exact output can pay much more than the quote they saw, whether because of ordinary trades ahead of theirs or a deliberate sandwich attack. The loss is bounded only by the user's balance and token approval. `sellPoolTokens` calls this function and inherits the problem. Combined with [H-1], users are also overcharged about 10x and have no way to notice or cap it.

Proof of Concept:

1. A user sees a quote for 10 WETH.
2. An attacker front-runs with a large `swapExactOutput`, moving the reserves so WETH costs more.
3. The user's transaction executes with no cap and pays more than quoted.
4. The attacker back-runs to restore the price and keeps the difference.

Proof Of Code

Place the following into `TSwapPool.t.sol`.

```
1  function testNoSlippageProtection() public {
2      vm.startPrank(liquidityProvider);
3      weth.approve(address(pool), 100e18);
4      poolToken.approve(address(pool), 100e18);
5      pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
```

```
6       vm.stopPrank();
7
8       uint256 quotedInput = pool.getInputAmountBasedOnOutput(
9           10e18, poolToken.balanceOf(address(pool)), weth.balanceOf(
                address(pool))
10      );
11
12      address attacker = makeAddr("attacker");
13      poolToken.mint(attacker, 10_000e18);
14      vm.startPrank(attacker);
15      poolToken.approve(address(pool), type(uint256).max);
16      pool.swapExactOutput(poolToken, weth, 30e18, uint64(block.timestamp
            ));
17      vm.stopPrank();
18
19      poolToken.mint(user, 10_000e18);
20      vm.startPrank(user);
21      poolToken.approve(address(pool), type(uint256).max);
22      uint256 balanceBefore = poolToken.balanceOf(user);
23      pool.swapExactOutput(poolToken, weth, 10e18, uint64(block.timestamp
            ));
24      vm.stopPrank();
25
26      uint256 paid = balanceBefore - poolToken.balanceOf(user);
27      assertGt(paid, quotedInput); // paid more than quoted, with no way
            to cap it
28  }
```

Recommended Mitigation: Add a `maxInputAmount` parameter and revert if the computed input exceeds it. Update `sellPoolTokens` and any other callers.

```
1     function swapExactOutput(
2         IERC20 inputToken,
3         IERC20 outputToken,
4         uint256 outputAmount,
5   +     uint256 maxInputAmount,
6         uint64 deadline
7     ) public ... {
8         ...
9         inputAmount = getInputAmountBasedOnOutput(outputAmount,
              inputReserves, outputReserves);
10  +     if (inputAmount > maxInputAmount) {
11  +         revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount);
12  +     }
13        _swap(inputToken, inputAmount, outputToken, outputAmount);
14    }
```

[H-3] `TSwapPool::sellPoolTokens` calls `swapExactOutput` with a pool-token amount as the WETH output, so users trade the wrong amount

Description: `sellPoolTokens` lets users sell `poolTokenAmount` pool tokens for WETH. It calls `swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount, uint64(block.timestamp))`. The third parameter of `swapExactOutput` is the exact amount of the output token (WETH) the caller wants, so the function asks for `poolTokenAmount` WETH and charges however many pool tokens that costs. A seller specifies an input amount, so the matching function is `swapExactInput`.

Impact: Callers receive a WETH amount equal to the number they passed in and pay an unpredictable number of pool tokens, often far more than they meant to sell. A user holding only the amount they intended to sell hits a failed transfer, so the function fails in the common case. The function also inherits [H-1] and [H-2]. Its return value is the pool tokens paid, not the WETH received that the name `wethAmount` implies. Its deadline is `block.timestamp`, so it can never be in the past and the deadline check does nothing.

Proof of Concept: In a pool with 100e18 pool tokens and 100e18 WETH, selling 10e18 pool tokens should return about 9.07e18 WETH. Instead the caller receives 10e18 WETH and pays well over 10e18 pool tokens.

Proof Of Code

Place the following into `TSwapPool.t.sol`.

```
function testSellPoolTokensTradesWrongAmount() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    poolToken.mint(user, 200e18);
    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    uint256 poolTokensBefore = poolToken.balanceOf(user);
    uint256 wethBefore = weth.balanceOf(user);
    pool.sellPoolTokens(10e18);
    vm.stopPrank();

    assertEq(weth.balanceOf(user) - wethBefore, 10e18);          // asked for 10 WETH out
    assertGt(poolTokensBefore - poolToken.balanceOf(user), 10e18); // paid far more than 10 pool tokens
}
```

Recommended Mitigation: Use `swapExactInput`, and give the function slippage and deadline parameters.

```
- function sellPoolTokens(uint256 poolTokenAmount) external returns (
    uint256 wethAmount) {
-     return swapExactOutput(i_poolToken, i_wethToken, poolTokenAmount,
     uint64(block.timestamp));
+ function sellPoolTokens(
+     uint256 poolTokenAmount,
+     uint256 minWethToReceive,
+     uint64 deadline
+ ) external returns (uint256 wethAmount) {
+     return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken,
    minWethToReceive, deadline);
  }
```

This fix also depends on [L-2]: until `swapExactInput` assigns its return value, `sellPoolTokens` would return 0.

[H-4] In `TSwapPool::_swap` the extra tokens paid out every 10 swaps break the `x * y = k` invariant and drain the pool

Description: The protocol relies on the constant-product invariant: the product of the two reserves (`x * y`) must not decrease across a swap, and grows slightly from fees. `_swap` breaks this with an extra incentive:

```
swap_count++;
if (swap_count >= SWAP_COUNT_MAX) {
    swap_count = 0;
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
}
```

On every 10th swap the pool sends the caller an extra 1e18 of the output token, on top of the `outputAmount` the swap already pays. The pool loses more of the output token than the pricing formula accounts for, so `x * y` falls. The payout is also an external call made before the swap's own transfers, which Slither reports as a reentrancy-events ordering issue.

Impact: Whoever makes the 10th swap receives 1 extra token from the pool's reserves, and the amount does not depend on the swap size. An attacker can repeat cheap swaps to collect it, and ordinary users trigger it too, so reserves fall even without an attacker. Liquidity providers lose value over time and the protocol's core invariant is broken. If the pool holds less than 1e18 of the output token, the 10th swap reverts, which blocks trading until reserves change.

Proof of Concept: The test performs 9 swaps, snapshots the pool's WETH, then makes the 10th swap. The pool should lose exactly the requested 1e17 WETH but loses 1.1e18. The scenario was found by the stateful fuzz suite and reduced to a unit test.

Proof Of Code

Place the following into `TSwapPool.t.sol`.

```
function testBonusPayoutBreaksInvariant() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputWeth = 1e17;

    vm.startPrank(user);
    poolToken.mint(user, 100e18);
    poolToken.approve(address(pool), type(uint256).max);

    for (uint256 i = 0; i < 9; i++) {
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
            timestamp));
    }

    int256 startingWeth = int256(weth.balanceOf(address(pool)));
    int256 expectedDelta = -int256(outputWeth);

    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
        timestamp)); // 10th swap
    vm.stopPrank();

    int256 actualDelta = int256(weth.balanceOf(address(pool))) -
        startingWeth;
    assertEq(actualDelta, expectedDelta); // fails: the pool also paid
        out the extra 1e18
}
```

Recommended Mitigation: Remove the extra incentive. If a trading incentive is wanted, fund it from outside the pool's reserves. Remove the unused counter and constant too, and consider a `k` check at the end of `_swap` so any future change that breaks the invariant reverts.

```
- uint256 private swap_count = 0;
- uint256 private constant SWAP_COUNT_MAX = 10;
  ...
- swap_count++;
- if (swap_count >= SWAP_COUNT_MAX) {
-     swap_count = 0;
```

```
7 -     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
8 - }
```

Medium

[M-1] `TSwapPool::deposit` never applies its deadline, so deposits can execute after the deadline the user set

Description: `deposit` accepts a `deadline` parameter documented as "The deadline for the transaction to be completed by", but never uses it. `withdraw` and the swap functions apply the `revertIfDeadlinePassed` modifier, so `deposit` is inconsistent with the rest of the pool.

Impact: A deposit that sits pending can execute long after the user intended, at whatever pool ratio exists then. The user's `minimumLiquidityTokensToMint` and `maximumPoolTokensToDeposit` still bound the worst case, which is why this is rated Medium. It still lets a searcher or the network delay a deposit until conditions suit someone else.

Proof of Concept: The compiler reports the unused parameter:

```
Warning (5667): Unused function parameter. Remove or comment out the
    variable name to silence this warning.
  --> src/TSwapPool.sol:96:9:
   |
96 |         uint64 deadline
   |         ^^^^^^^^^^^^^^^
```

Proof Of Code

Place the following into `TSwapPool.t.sol`.

```
function testDepositIgnoresDeadline() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);

    uint64 deadline = uint64(block.timestamp);
    vm.warp(block.timestamp + 1 days); // the deadline is now in the
        past

    // expected: revert with TSwapPool__DeadlineHasPassed, actual:
        succeeds
    pool.deposit(100e18, 100e18, 100e18, deadline);
    vm.stopPrank();

    assertEq(pool.balanceOf(liquidityProvider), 100e18);
```

```
14 }
```

Recommended Mitigation:

```
1    function deposit(
2        uint256 wethToDeposit,
3        uint256 minimumLiquidityTokensToMint,
4        uint256 maximumPoolTokensToDeposit,
5        uint64 deadline
6    )
7        external
8  +     revertIfDeadlinePassed(deadline)
9        revertIfZero(wethToDeposit)
10       returns (uint256 liquidityTokensToMint)
```

Low

[L-1] `TSwapPool::LiquidityAdded` event emits its amounts in the wrong order

Description: `_addLiquidityMintAndTransfer` emits `LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit)`, but the event is declared as `LiquidityAdded(address indexed liquidityProvider, uint256 wethDeposited, uint256 poolTokensDeposited)`. The second and third arguments are swapped.

Impact: Every deposit logs WETH as pool tokens and pool tokens as WETH, so off-chain consumers such as subgraphs, dashboards, and bots read wrong data. No funds are directly at risk.

Recommended Mitigation:

```
1  - emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
2  + emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] `TSwapPool::swapExactInput` never assigns its named return value, so it always returns 0

Description: `swapExactInput` is expected to return the amount of tokens the caller bought. It declares `returns (uint256 output)` but computes a separate local `outputAmount` and never assigns `output` or uses a **`return`** statement, so Solidity returns the default value.

Impact: The return value is always 0. Any integration or contract that reads it gets wrong information, including a corrected `sellPoolTokens` (see [H-3]).

Proof of Concept:

Proof Of Code

Place the following into `TSwapPool.t.sol`.

```
function testSwapExactInputReturnsZero() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    uint256 returned = pool.swapExactInput(poolToken, 1e18, weth, 0,
        uint64(block.timestamp));
    vm.stopPrank();

    assertEq(returned, 0); // the swap succeeded but reports 0 output
}
```

Recommended Mitigation:

```
- uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount,
    inputReserves, outputReserves);
+ output = getOutputAmountBasedOnInput(inputAmount, inputReserves,
    outputReserves);

- if (outputAmount < minOutputAmount) {
-     revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
+ if (output < minOutputAmount) {
+     revert TSwapPool__OutputTooLow(output, minOutputAmount);
  }

- _swap(inputToken, inputAmount, outputToken, outputAmount);
+ _swap(inputToken, inputAmount, outputToken, output);
```

[L-3] `TSwapPool::getPriceOfOnePoolTokenInWeth` hardcodes `1e18` as one pool token, giving meaningless prices for tokens that do not use 18 decimals

Description: `getPriceOfOnePoolTokenInWeth` passes `1e18` as the input amount. That equals one token only if the pool token has 18 decimals. WETH has 18 decimals, so `getPriceOfOneWethInPoolTokens` is correct.

Impact: For a pool token with 6 decimals (for example USDC), `1e18` is 10^12 whole tokens, so the function returns nearly the entire WETH reserve instead of a price. Integrations that read it get unusable values.

Recommended Mitigation: Use the pool token's own decimals.

```
+ uint256 oneToken = 10 ** IERC20Metadata(address(i_poolToken)).decimals();
  return getOutputAmountBasedOnInput(
-     1e18, i_poolToken.balanceOf(address(this)), i_wethToken.balanceOf(address(this))
+     oneToken, i_poolToken.balanceOf(address(this)), i_wethToken.balanceOf(address(this))
  );
```

Informationals

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed

The error is declared but never used, so `getPool` and `getToken` silently return `address(0)` for unknown entries.

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] `PoolFactory::constructor` lacks a zero-address check on `wethToken`

`i_wethToken` is immutable, so a zero address leaves a factory whose pools can never work and requires redeployment.

```
+ error PoolFactory__ZeroAddress();
  constructor(address wethToken) {
+     if (wethToken == address(0)) revert PoolFactory__ZeroAddress();
      i_wethToken = wethToken;
  }
```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`

The LP token symbol is built from the token's name, producing long symbols such as `tsWrapped Bitcoin` instead of `tsWBTC`. This can break UIs and integrations that expect a short ticker.

```
- string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).name());
+ string memory liquidityTokenSymbol = string.concat("ts", IERC20(
    tokenAddress).symbol());
```

[I-4] `TSwapPool::constructor` lacks zero-address checks on `poolToken` and `wethToken`

Pools normally come from the factory, but a direct deployment with a zero address creates a permanently broken pool.

```
+ error TSwapPool__ZeroAddress();
  ...
+ if (wethToken == address(0) || poolToken == address(0)) revert
    TSwapPool__ZeroAddress();
  i_wethToken = IERC20(wethToken);
  i_poolToken = IERC20(poolToken);
```

[I-5] Events are missing `indexed` fields

Indexed event fields let off-chain tools filter logs cheaply. `Swap`, `LiquidityAdded`, `LiquidityRemoved` in `TSwapPool.sol` and `PoolCreated` in `PoolFactory.sol` index few or none of their parameters. Each indexed field costs extra gas and an event can index at most three, so index the fields integrators filter on. This item is a matter of preference.

```
- event Swap(address indexed swapper, IERC20 tokenIn, uint256
    amountTokenIn, IERC20 tokenOut, uint256 amountTokenOut);
+ event Swap(address indexed swapper, IERC20 indexed tokenIn, uint256
    amountTokenIn, IERC20 indexed tokenOut, uint256 amountTokenOut);
```

[I-6] Magic numbers should be replaced with named constants

`TSwapPool` uses unexplained literals such as `997`, `1000`, `10000`, and `1e18` in its pricing and payout logic. The `10000` typo in [H-1] is the kind of mistake named constants make easy to spot.

```
+ uint256 private constant FEE_NUMERATOR = 997;
+ uint256 private constant FEE_DENOMINATOR = 1000;
- uint256 inputAmountMinusFee = inputAmount * 997;
+ uint256 inputAmountMinusFee = inputAmount * FEE_NUMERATOR;
```

[I-7] Dead and redundant code in `TSwapPool::deposit`

- The local variable `poolTokenReserves` in the existing-liquidity branch is never used.
- `TSwapPool__WethDepositAmountTooLow` is raised with `MINIMUM_WETH_LIQUIDITY`, a constant anyone can read, so the argument adds nothing.
- In the first-deposit branch, `liquidityTokensToMint = wethToDeposit;` runs after the external calls in `_addLiquidityMintAndTransfer`. It is a local variable, so there is no risk, but assigning it first would follow checks-effects-interactions.

```
1 - uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

[I-8] NATSPEC is missing or incomplete on the swap functions

`swapExactInput` has no NATSPEC. The comment on `swapExactOutput` begins with `/*`, which documentation tools ignore (use `/**` or `///`), and it omits the `deadline` parameter.

[I-9] `TSwapPool::swapExactInput` could be marked `external`

Nothing calls `swapExactInput` internally today, so it could be `external`. If [H-3] is fixed by having `sellPoolTokens` call it internally, it must stay **`public`**.

[I-10] `PoolFactory` imports a testing interface and uses an invalid SPDX identifier

`PoolFactory` imports `IERC20` from `forge-std/interfaces/IERC20.sol`, a testing dependency; use OpenZeppelin's `IERC20Metadata` (which also covers `name()` and `symbol()`). The license identifier `GNU General Public License v3.0` is not a valid SPDX string; use `GPL-3.0-only` or `GPL-3.0-or-later`.

[I-11] Missing validation in `getInputAmountBasedOnOutput` and the swap functions

`getInputAmountBasedOnOutput` never checks `outputAmount < outputReserves`. Equal values give a zero denominator and a larger amount underflows, so the call reverts with a raw panic instead of a clear custom error. `swapExactInput` and `swapExactOutput` also read reserves from caller-supplied token addresses before `_swap` rejects unknown tokens. Validate the tokens and amounts first and revert with custom errors.

[I-12] Price getters return fee-adjusted output rather than a spot price

`getPriceOfOneWethInPoolTokens` and `getPriceOfOnePoolTokenInWeth` call `getOutputAmountBasedOnInput`, so the result includes the 0.3% fee and price impact. They are derived from current reserves, so a protocol that uses them as an oracle can be manipulated within one transaction, for example with a flash loan. They also revert on an empty pool because of `revertIfZero(outputReserves)`. Document these properties in the NATSPEC and warn integrators.

[I-13] Fee-on-transfer and rebasing tokens are not supported

The pool computes prices from `balanceOf` reserves and assumes transfers move the stated amount. Fee-on-transfer and rebasing tokens break that assumption, and `SafeERC20` does not address it (it handles missing or false return values only). Document the supported token types or reject unsupported tokens when creating a pool.